

Agentic AI Internship Curriculum

Building AI agents with large language models - 4, 8 and 12 week paths

DawoodTech NextGen

25 September 2026

Contents

1. Overview
2. 4-Week Path: Summary
3. 8-Week Path: Summary
4. 12-Week Path: Summary
5. 4-Week Path: Week by Week
6. 8-Week Path: Week by Week
7. 12-Week Path: Week by Week

Overview

Who this is for

Interns who want to build real AI agents: programs where a large language model plans, calls tools, uses documents and works with other agents to finish a task. Each week combines learning tasks (read, try, explain) with a practical project reviewed by a supervisor.

Prerequisites

- Basic Python: functions, lists/dicts, classes, virtual environments
- Has called a web API before (HTTP, JSON)
- Git and a GitHub account
- A computer that can run Python 3.11+, Node.js LTS and Docker (Docker needed from the browser/code week)

Tools and accounts

Tool / account	Used for	API key / cost
Google AI Studio (Gemini API)	Main model provider during the course	API key needed - free tier with rate limits
Ollama	Local models for offline testing	No key - free, runs locally
uv, Python 3.11+	Project and package management	Free
Chroma	Vector database for RAG	Free, local
MCP Python SDK + MCP Inspector	Building and testing MCP servers	Free
LangGraph / OpenAI Agents SDK / Pydantic AI / Google ADK	Agent frameworks (choose one or two)	Free libraries; model key needed
Playwright MCP / Browser Use	Browser agents	Free libraries
Docker	Sandboxed code execution, deployment	Free (Docker Desktop has licence terms for large companies)
E2B (optional)	Hosted code sandboxes	API key needed
promptfoo	Evals and red-teaming	Free, open source
Langfuse	Tracing and observability	Free self-hosted or cloud free tier (key needed)
Hugging Face Spaces / Render	Free deployment for demos	Free account

Free tiers change. Check current limits before starting, and never send private or customer data to a free tier.

How weekly review works

1. At the start of the week, the week's task is assigned in TaskDesk and unlocks on the intern's roadmap.
2. The intern completes the learning tasks and the project during the week.
3. The intern submits a public GitHub repository URL and a demo (live URL or screen-recorded video), plus the week's other deliverables.
4. The supervisor reviews against the checklist and approves, or returns the task with "needs improvement" notes.
5. When the supervisor approves, the next week's task is assigned automatically and the next week unlocks.

Choosing a path

Path	Pace	Best for
4 weeks	Intensive: 3 topics per week	Interns with strong Python who want a fast overview and one deployed project
8 weeks	Balanced: 1-2 topics per week	Most interns
12 weeks	Deep: 1 topic per week	Interns aiming for a portfolio-grade capstone and job readiness

4-Week Path: Summary

Week	Topic	Project	Key deliverable
1	LLM Foundations, Tool Calling and Memory	Personal Assistant: Tools, Structured Output and Memory	Before/after token measurement table
2	RAG, Agentic RAG and MCP	Agentic RAG Assistant Served Over MCP	Inspector screenshots for each tool
3	Frameworks, Multi-Agent Systems, Web and Code Tools	Framework Multi-Agent Team With Web and Code Tools	Architecture diagram
4	Evals, Security and Production	Capstone: Evaluated, Secured and Deployed Agent	Live URL

8-Week Path: Summary

Week	Topic	Project	Key deliverable
1	LLM Foundations and the Agent Loop	Tool-Using Assistant With Structured Output	<code>tests/questions.md</code> with 15 questions and the tools used for each
2	Context, Memory and RAG	Document Assistant With Memory	<code>eval/questions.csv</code> with question, expected source and result
3	Agentic RAG and MCP	Agentic RAG Assistant Served Over MCP	Inspector screenshots for each tool
4	Agent Frameworks and Human-in-the-Loop	Rebuild With a Framework + Human Approval	Comparison: plain loop vs framework (lines of code, features, debugging)
5	Multi-Agent Systems	Multi-Agent Research Team	Architecture diagram
6	Browser and Code-Executing Agents	Web Data + Code Analysis Agent	Chart output and summary for one real dataset
7	Evaluation, Observability and Security	Evaluate, Trace and Secure Your Agent	Eval report: pass rate, cost/task, p50/p95 latency
8	Production and Capstone	Capstone: Ship a Production Agent	Live URL

12-Week Path: Summary

Week	Topic	Project	Key deliverable
1	LLM Foundations and Structured Output	Smart Text Extractor CLI	<code>samples/</code> folder with 10 inputs and their outputs
2	Tool Calling and Your First Agent Loop	Tool-Using Assistant From Scratch	<code>tests/questions.md</code> with 15 questions and the tools used for each
3	Tool Design, Context Engineering and Memory	Personal Assistant With Memory	Before/after token measurement table
4	RAG: Answering From Your Documents	Document Q&A Bot With Citations	<code>eval/questions.csv</code> with question, expected source and result
5	Agentic RAG	Agentic Research Assistant	Table of 15 test questions with searches used and correctness
6	Model Context Protocol (MCP)	Your Own MCP Server	Inspector screenshots for each tool
7	Agent Frameworks and Human-in-the-Loop	Rebuild With a Framework + Human Approval	Comparison: plain loop vs framework (lines of code, features, debugging)
8	Multi-Agent Systems	Multi-Agent Research Team	Architecture diagram
9	Browser and Code-Executing Agents	Web Data + Code Analysis Agent	Chart output and summary for one real dataset
10	Evaluation and Observability	Evaluate and Trace Your Agent	Eval report: pass rate, cost/task, p50/p95 latency
11	Agent Safety and Security	Red-Team and Harden Your Agent	Threat model table
12	Production and Capstone	Capstone: Ship a Production Agent	Live URL

Week 1: LLM Foundations, Tool Calling and Memory

Objective

Call a large language model (LLM) from Python, control it with good prompts, and get reliable structured JSON back that your code can trust. Give a model tools (Python functions) and build the agent loop yourself: the model picks a tool, your code runs it, the result goes back, until the task is done. Design tools that agents use well, manage what goes into the context window, and give the agent short-term and simple long-term memory.

Main topics

- What an LLM is and what makes a program an "agent"
- Calling a model API from Python (OpenAI-compatible APIs, Gemini free tier, local models with Ollama)
- Prompting for agents: system prompts, roles, few-shot examples
- Function / tool calling: how a model asks your code to run a function
- The agent loop: reason, act, observe, repeat
- Stop conditions: final answer, step limit, errors
- Designing good tools: naming, descriptions, inputs, outputs and errors
- Context engineering: choosing what the model sees on each call
- Short-term memory: conversation history, trimming and summarising

Detailed concepts

- Tokens, context window, temperature and max output tokens
- Messages array: system, user and assistant roles
- Keeping API keys in a `.env` file, never in code
- Tool schema: name, description and JSON-schema parameters
- Tool call IDs and sending tool results back with the `tool` role
- Parallel tool calls in one response
- Few, well-scoped tools instead of many overlapping ones
- Token-efficient tool results (pagination, filtering, summaries)
- Context rot: quality drops as context grows

Learning tasks

1. Create a free Google AI Studio key and run your first chat completion from Python using the OpenAI-compatible endpoint.
2. Read "Building effective agents" and write a half-page summary of the difference between workflows and agents.
3. Read "Writing effective tools for agents" and list five rules you will apply to your own tools.
4. Read the Gemini "OpenAI compatibility" page and note which parameters are supported.
5. Skim the ReAct paper abstract and figure 1; explain the reason, act, observe cycle in your own words.

Resources

- Gemini API documentation
<https://ai.google.dev/gemini-api/docs>
- Gemini OpenAI compatibility
<https://ai.google.dev/gemini-api/docs/openai>
- Anthropic - Building effective agents
<https://www.anthropic.com/engineering/building-effective-agents>
- Gemini API - Function calling
<https://ai.google.dev/gemini-api/docs/function-calling>
- Anthropic - Writing effective tools for agents
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- Anthropic - Effective context engineering for AI agents
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Practical project: Personal Assistant: Tools, Structured Output and Memory

In one week, build a complete personal assistant: provider-agnostic API calls with validated JSON, your own agent loop with 3-4 well-designed tools, conversation compaction, and long-term memory in SQLite that survives restarts.

1. Setup: uv, `.env`, provider config, first validated JSON call.
2. Tools (weather, calculator, notes) and your own agent loop with step limit and logging.
3. Memory tools (`remember` , `recall`) in SQLite keyed per user; load memories at start.
4. History compaction with a before/after token measurement.
5. AGENTS.md, 15 test questions, README and demo.

Bonus: Add a `/forget` command that lets the user see and delete stored memories.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Before/after token measurement table
3. `AGENTS.md` at the repo root
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Can call any OpenAI-compatible model API from Python and switch providers with configuration only
- Writes clear system prompts with rules, examples and an output format
- Can write tool schemas the model uses correctly
- Can build and debug an agent loop without any framework
- Designs tools with clear contracts and compact, useful results

- Keeps long conversations within budget without losing key facts

Week 2: RAG, Agentic RAG and MCP

Objective

Let an agent answer questions from your own documents by embedding them, storing them in a vector database, retrieving the best chunks and citing them. Turn a fixed RAG pipeline into an agent that decides when and what to search, rewrites queries, combines several sources and checks its own answer. Build an MCP server that exposes your tools and data to any MCP-compatible AI app, test it with the MCP Inspector, and call it from a client.

Main topics

- Embeddings: turning text into vectors that capture meaning
- Vector databases: Chroma locally, pgvector in PostgreSQL
- The RAG pipeline: load, chunk, embed, store, retrieve, generate
- From RAG pipeline to agentic RAG: retrieval as a tool
- Query rewriting and breaking a question into sub-questions
- Multiple sources: your documents, the web (Wikipedia), and user memory
- What MCP is and why it exists (one standard instead of custom integrations)
- MCP building blocks: tools, resources and prompts
- Transports: stdio for local, Streamable HTTP for remote servers

Detailed concepts

- Cosine similarity and nearest-neighbour search
- Chunk size and overlap, and why they matter
- Metadata (source file, page) for filtering and citations
- Retrieval tools with filters (source, date, section)
- Multi-hop questions that need several searches
- Contextual retrieval: adding context to chunks before embedding
- Hosts, clients and servers
- Tool schemas generated from Python type hints
- Testing with the MCP Inspector

Learning tasks

1. Read the embeddings guide for your provider and embed ten sentences; find the most similar pair.
2. Read "Introducing Contextual Retrieval" and list the two techniques it combines.
3. Read the MCP introduction and architecture pages; draw the host-client-server diagram.
4. Follow the Chroma getting-started guide and store/query your ten sentences with metadata.
5. Turn your `retrieve()` function into a tool with parameters `query`, `source` and `k`, and let the agent call it.

Resources

- Gemini API - Embeddings
<https://ai.google.dev/gemini-api/docs/embeddings>
- Chroma documentation
<https://docs.trychroma.com/>
- Anthropic - Introducing Contextual Retrieval
<https://www.anthropic.com/news/contextual-retrieval>
- Chroma - metadata filtering
<https://docs.trychroma.com/>
- Model Context Protocol - documentation
<https://modelcontextprotocol.io/>
- MCP Specification 2026-07-28
<https://modelcontextprotocol.io/specification/2026-07-28>

Practical project: Agentic RAG Assistant Served Over MCP

Build a document assistant with agentic retrieval (retrieval as a tool, query rewriting, citations) and expose its search and notes as an MCP server that works in the MCP Inspector and in a real MCP host.

1. Ingest documents into Chroma with metadata; implement `search_docs` as a tool.
2. Agent rules: query rewriting, citations, max 3 searches, refuse when not found.
3. Create an MCP server exposing `search_docs`, `add_note`, `list_notes`.
4. Test in the Inspector and in one MCP host.
5. Test 15 questions (incl. multi-hop and unanswerable) and document results.

Bonus: Expose the documents as MCP resources and add one prompt template.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Inspector screenshots for each tool
3. Host configuration snippet in the README
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds a complete RAG pipeline from files to cited answers
- Chooses chunking and retrieval settings based on measurement, not guesses
- Builds an agent that plans its own retrieval steps
- Handles multi-hop questions with query rewriting
- Explains MCP hosts, clients, servers, tools, resources and prompts
- Ships a working, documented MCP server

Week 3: Frameworks, Multi-Agent Systems, Web and Code Tools

Objective

Rebuild your agent with a current framework, use explicit state, checkpoints and human approval steps, and decide when a framework is worth it. Design and build a system where several specialised agents cooperate - an orchestrator with workers, handoffs between agents and a human approval step - and understand agent-to-agent protocols. Give agents safe access to a real browser and to code execution in a sandbox, so they can collect data from websites and analyse it with code.

Main topics

- The current framework landscape: LangGraph, OpenAI Agents SDK, Pydantic AI, Google ADK, CrewAI
- Graphs and state machines for agents
- Checkpointing and resuming long-running agents
- When multiple agents help (and when they only add cost)
- Patterns: orchestrator-worker, handoffs, parallel workers, reviewer/critic
- Planning: breaking a goal into subtasks
- Browser agents: Playwright MCP and Browser Use
- Accessibility tree vs screenshots (how agents "see" a page)
- Computer-use agents that control a desktop, and their risks

Detailed concepts

- Nodes, edges and conditional routing
- Typed state objects
- Threads and persistence
- Clear roles and narrow tools per agent
- Passing context between agents (what to share, what to hide)
- Parallel subagents and merging results
- Page snapshots, element references and actions (click, type, navigate)
- Headless vs headed browsers
- Why generated code must never run on your own machine unprotected

Learning tasks

1. Read the overview pages of LangGraph and the OpenAI Agents SDK, and fill a comparison table (model support, state, HITL, tracing, learning curve).
2. Read "How we built our multi-agent research system" and note three lessons about orchestration and cost.
3. Run Playwright MCP (`npx @playwright/mcp@latest`) in an MCP host and let an agent find and summarise a public page.
4. Do the LangGraph quickstart and add a conditional edge.
5. Read the OpenAI Agents SDK handoffs page and run the handoff example.

Resources

- LangGraph documentation
<https://docs.langchain.com/oss/python/langgraph/overview>
- OpenAI Agents SDK (Python)
<https://openai.github.io/openai-agents-python/>
- Anthropic - How we built our multi-agent research system
<https://www.anthropic.com/engineering/multi-agent-research-system>
- OpenAI Agents SDK - Handoffs
<https://openai.github.io/openai-agents-python/handoffs/>
- Playwright MCP (Microsoft)
<https://github.com/microsoft/playwright-mcp>
- Browser Use
<https://github.com/browser-use/browser-use>

Practical project: Framework Multi-Agent Team With Web and Code Tools

Using a current framework, build a research team (orchestrator, researcher, analyst, writer) where the researcher can browse public pages via Playwright MCP and the analyst runs Python in a no-network Docker sandbox, with a human approval step before the final report is saved.

1. Pick the framework, design the agents and the approval point (diagram).
2. Researcher with browsing (Playwright MCP) restricted to public pages and an allow-list.
3. Analyst that writes Python and runs it only in the Docker sandbox (no network, limits).
4. Writer + orchestrator with parallel research and persistent state.
5. Human approval before saving; log tokens and actions per agent.

Bonus: Expose the research team as a service with an A2A-style Agent Card describing its skill.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Architecture diagram
3. Two example briefs with sources
4. Token/cost per agent for one run
5. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds a stateful agent graph with conditional routing
- Adds checkpoints and human approval to an agent
- Chooses a multi-agent pattern that fits the problem
- Builds an orchestrator with specialised workers and a review step
- Builds an agent that navigates websites through a browser tool

- Runs model-written code only inside an isolated sandbox with limits

Week 4: Evals, Security and Production

Objective

Measure how good your agent is with a test set and automated evals, trace every run, and report quality, cost and latency with numbers. Find and fix the main security risks of agents - prompt injection, tool misuse, excessive permissions, data leaks - using the OWASP Top 10 for Agentic Applications and red-team tests. Package the agent as a real service with an API and UI, streaming, retries, caching and cost limits, deploy it, and present a portfolio-grade capstone.

Main topics

- Why agents need evals: non-deterministic output, many steps
- Test sets, graders (code checks, LLM-as-judge, human review)
- Evaluating the path (trajectory) as well as the final answer
- OWASP Top 10 for Agentic Applications (2026)
- Direct and indirect prompt injection
- Least privilege, tool permissions and human approval
- Serving an agent with FastAPI and streaming responses
- Reliability: timeouts, retries with backoff, fallbacks, rate limits
- Cost control: caching, model routing, budgets

Detailed concepts

- Golden datasets and edge cases
- Deterministic assertions vs model-graded rubrics
- Pass rate, cost per task, latency (p50/p95)
- Goal hijack, tool misuse, identity and privilege abuse
- The "lethal trifecta": private data + untrusted content + a way to send data out
- Input/output guardrails and allow-lists
- Server-sent events (SSE) for token streaming
- Idempotency and background jobs for long tasks
- Prompt caching and response caching

Learning tasks

1. Read "Demystifying evals for AI agents" and summarise the grader types it describes.
2. Read the OWASP Top 10 for Agentic Applications and map at least 5 risks to your own agent.
3. Follow the FastAPI tutorial up to "Request Body" and build a `/chat` endpoint that calls your agent.
4. Install promptfoo and run its getting-started example.
5. Read "The lethal trifecta" article and check whether your agent has all three parts.

Resources

- Anthropic - Demystifying evals for AI agents
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

- promptfoo documentation
<https://www.promptfoo.dev/docs/intro/>
- OWASP Top 10 for Agentic Applications for 2026
<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- OWASP Top 10 for LLM Applications
<https://genai.owasp.org/llm-top-10/>
- FastAPI documentation
<https://fastapi.tiangolo.com/>
- Docker - Get started
<https://docs.docker.com/get-started/>

Practical project: Capstone: Evaluated, Secured and Deployed Agent

Ship your agent as a product: eval set and tracing, a quick security hardening pass, a FastAPI service with streaming and a simple UI, Docker packaging and deployment to a free host, plus a final README and demo video.

1. 20+ eval cases with one-command runner; tracing on.
2. Threat model + 10 attack cases; fix the top issues in code (approvals, allow-lists).
3. FastAPI `/health` + streaming `/chat`, simple UI, retries and cost limits.
4. Docker + deploy to a free host with secrets as env vars.
5. Final README (architecture, eval numbers, security notes) and 3-5 minute video.

Bonus: Add a background-job endpoint for long tasks with job status polling.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Live URL
3. 3-5 minute demo video
4. Final README: architecture, evals, security, next steps
5. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds and maintains a test set for an agent
- Runs automated evals and reads the results critically
- Threat-models an agent using the OWASP agentic risks
- Defends against prompt injection with design, not just prompts
- Ships an agent as a deployed, documented web service
- Handles failures, limits and costs deliberately

Week 1: LLM Foundations and the Agent Loop

Objective

Call a large language model (LLM) from Python, control it with good prompts, and get reliable structured JSON back that your code can trust. Give a model tools (Python functions) and build the agent loop yourself: the model picks a tool, your code runs it, the result goes back, until the task is done.

Main topics

- What an LLM is and what makes a program an "agent"
- Calling a model API from Python (OpenAI-compatible APIs, Gemini free tier, local models with Ollama)
- Prompting for agents: system prompts, roles, few-shot examples
- Function / tool calling: how a model asks your code to run a function
- The agent loop: reason, act, observe, repeat
- Stop conditions: final answer, step limit, errors

Detailed concepts

- Tokens, context window, temperature and max output tokens
- Messages array: system, user and assistant roles
- Keeping API keys in a `.env` file, never in code
- Tool schema: name, description and JSON-schema parameters
- Tool call IDs and sending tool results back with the `tool` role
- Parallel tool calls in one response

Learning tasks

1. Create a free Google AI Studio key and run your first chat completion from Python using the OpenAI-compatible endpoint.
2. Read "Building effective agents" and write a half-page summary of the difference between workflows and agents.
3. Read the Gemini "OpenAI compatibility" page and note which parameters are supported.
4. Skim the ReAct paper abstract and figure 1; explain the reason, act, observe cycle in your own words.
5. Write the same request with three different system prompts and compare the answers in a short table.

Resources

- Gemini API documentation
<https://ai.google.dev/gemini-api/docs>
- Gemini OpenAI compatibility
<https://ai.google.dev/gemini-api/docs/openai>

- Anthropic - Building effective agents
<https://www.anthropic.com/engineering/building-effective-agents>
- Gemini API - Function calling
<https://ai.google.dev/gemini-api/docs/function-calling>

Practical project: Tool-Using Assistant With Structured Output

A terminal assistant built on your own agent loop with at least three tools (weather via Open-Meteo, calculator, SQLite notes), a strict Pydantic-validated JSON summary at the end of each answer, and configuration that works with the Gemini free tier or Ollama.

1. Set up uv, `.env.example` and provider configuration.
2. Write and test three tools, then their schemas.
3. Implement the agent loop with step limit, error handling and logging.
4. Return a final `AnswerSummary` object validated with Pydantic (answer, tools_used, confidence).
5. Test with 15 questions and record tools used.

Bonus: Generate tool schemas automatically from the function signatures and docstrings.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. `tests/questions.md` with 15 questions and the tools used for each
3. Log output of three runs in the README
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Can call any OpenAI-compatible model API from Python and switch providers with configuration only
- Writes clear system prompts with rules, examples and an output format
- Can write tool schemas the model uses correctly
- Can build and debug an agent loop without any framework

Week 2: Context, Memory and RAG

Objective

Design tools that agents use well, manage what goes into the context window, and give the agent short-term and simple long-term memory. Let an agent answer questions from your own documents by embedding them, storing them in a vector database, retrieving the best chunks and citing them.

Main topics

- Designing good tools: naming, descriptions, inputs, outputs and errors
- Context engineering: choosing what the model sees on each call
- Short-term memory: conversation history, trimming and summarising
- Embeddings: turning text into vectors that capture meaning
- Vector databases: Chroma locally, pgvector in PostgreSQL
- The RAG pipeline: load, chunk, embed, store, retrieve, generate

Detailed concepts

- Few, well-scoped tools instead of many overlapping ones
- Token-efficient tool results (pagination, filtering, summaries)
- Context rot: quality drops as context grows
- Cosine similarity and nearest-neighbour search
- Chunk size and overlap, and why they matter
- Metadata (source file, page) for filtering and citations

Learning tasks

1. Read "Writing effective tools for agents" and list five rules you will apply to your own tools.
2. Read the embeddings guide for your provider and embed ten sentences; find the most similar pair.
3. Read "Effective context engineering for AI agents" and explain context rot in three sentences.
4. Follow the Chroma getting-started guide and store/query your ten sentences with metadata.
5. Read the AGENTS.md site and the Agent Skills specification; write an AGENTS.md for your own project.

Resources

- Anthropic - Writing effective tools for agents
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- Anthropic - Effective context engineering for AI agents
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Gemini API - Embeddings
<https://ai.google.dev/gemini-api/docs/embeddings>

- Chroma documentation
<https://docs.trychroma.com/>

Practical project: Document Assistant With Memory

Extend your assistant with RAG over real documents (persistent Chroma index, citations, refusals) and long-term user memory in SQLite, with conversation compaction to keep token use under control.

1. Ingest 5-10 documents with chunking and metadata.
2. Grounded answers with citations and refusals.
3. Memory tools keyed per user; load memories at session start.
4. History compaction with a before/after token measurement.
5. 20 test questions incl. unanswerable ones; simple UI.

Bonus: Add hybrid search (keyword + vector) and compare results on names or codes.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. `eval/questions.csv` with question, expected source and result
3. Chunk-size comparison table in the README
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Designs tools with clear contracts and compact, useful results
- Keeps long conversations within budget without losing key facts
- Builds a complete RAG pipeline from files to cited answers
- Chooses chunking and retrieval settings based on measurement, not guesses

Week 3: Agentic RAG and MCP

Objective

Turn a fixed RAG pipeline into an agent that decides when and what to search, rewrites queries, combines several sources and checks its own answer. Build an MCP server that exposes your tools and data to any MCP-compatible AI app, test it with the MCP Inspector, and call it from a client.

Main topics

- From RAG pipeline to agentic RAG: retrieval as a tool
- Query rewriting and breaking a question into sub-questions
- Multiple sources: your documents, the web (Wikipedia), and user memory
- What MCP is and why it exists (one standard instead of custom integrations)
- MCP building blocks: tools, resources and prompts
- Transports: stdio for local, Streamable HTTP for remote servers

Detailed concepts

- Retrieval tools with filters (source, date, section)
- Multi-hop questions that need several searches
- Contextual retrieval: adding context to chunks before embedding
- Hosts, clients and servers
- Tool schemas generated from Python type hints
- Testing with the MCP Inspector

Learning tasks

1. Read "Introducing Contextual Retrieval" and list the two techniques it combines.
2. Read the MCP introduction and architecture pages; draw the host-client-server diagram.
3. Turn your `retrieve()` function into a tool with parameters `query`, `source` and `k`, and let the agent call it.
4. Read the 2026-07-28 specification release post and list three changes.
5. Write 10 multi-hop questions (needing 2+ searches) and record how many searches your agent makes for each.

Resources

- Anthropic - Introducing Contextual Retrieval
<https://www.anthropic.com/news/contextual-retrieval>
- Chroma - metadata filtering
<https://docs.trychroma.com/>
- Model Context Protocol - documentation
<https://modelcontextprotocol.io/>

- MCP Specification 2026-07-28
<https://modelcontextprotocol.io/specification/2026-07-28>

Practical project: Agentic RAG Assistant Served Over MCP

Build a document assistant with agentic retrieval (retrieval as a tool, query rewriting, citations) and expose its search and notes as an MCP server that works in the MCP Inspector and in a real MCP host.

1. Ingest documents into Chroma with metadata; implement `search_docs` as a tool.
2. Agent rules: query rewriting, citations, max 3 searches, refuse when not found.
3. Create an MCP server exposing `search_docs`, `add_note`, `list_notes`.
4. Test in the Inspector and in one MCP host.
5. Test 15 questions (incl. multi-hop and unanswerable) and document results.

Bonus: Expose the documents as MCP resources and add one prompt template.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Inspector screenshots for each tool
3. Host configuration snippet in the README
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds an agent that plans its own retrieval steps
- Handles multi-hop questions with query rewriting
- Explains MCP hosts, clients, servers, tools, resources and prompts
- Ships a working, documented MCP server

Week 4: Agent Frameworks and Human-in-the-Loop

Objective

Rebuild your agent with a current framework, use explicit state, checkpoints and human approval steps, and decide when a framework is worth it.

Main topics

- The current framework landscape: LangGraph, OpenAI Agents SDK, Pydantic AI, Google ADK, CrewAI
- Graphs and state machines for agents
- Checkpointing and resuming long-running agents
- Human-in-the-loop: pausing for approval or input

Detailed concepts

- Nodes, edges and conditional routing
- Typed state objects
- Threads and persistence
- Interrupts and resume commands
- Framework vs plain loop trade-offs

Learning tasks

1. Read the overview pages of LangGraph and the OpenAI Agents SDK, and fill a comparison table (model support, state, HITL, tracing, learning curve).
2. Do the LangGraph quickstart and add a conditional edge.
3. Add an interrupt before a risky tool call and resume it with an approval.
4. Write one page: "Would I use a framework for my project? Why or why not?"

Resources

- LangGraph documentation
<https://docs.langchain.com/oss/python/langgraph/overview>
- OpenAI Agents SDK (Python)
<https://openai.github.io/openai-agents-python/>
- Pydantic AI
<https://pydantic.dev/docs/ai/overview/>
- Google Agent Development Kit (ADK)
<https://google.github.io/adk-docs/>
- CrewAI documentation
<https://docs.crewai.com/>

Practical project: Rebuild With a Framework + Human Approval

Rebuild your research assistant with a current agent framework (LangGraph recommended; OpenAI Agents SDK, Pydantic AI or Google ADK allowed), with explicit state, a persistent checkpoint, and a human approval step before any action that changes data or sends something.

1. Choose a framework and justify it in the README.
2. Define the state schema and the graph/agents.
3. Plug in your existing tools (or your MCP server).
4. Add a SQLite (or other persistent) checkpoint and resume a conversation after restart.
5. Add an approval interrupt before `add_note` /any write action and resume with approve/reject.

Bonus: Stream intermediate steps (tool calls) to the terminal or UI while the agent runs.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Comparison: plain loop vs framework (lines of code, features, debugging)
3. Demo of approve and reject paths
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds a stateful agent graph with conditional routing
- Adds checkpoints and human approval to an agent
- Makes a reasoned framework choice for a project

Week 5: Multi-Agent Systems

Objective

Design and build a system where several specialised agents cooperate - an orchestrator with workers, handoffs between agents and a human approval step - and understand agent-to-agent protocols.

Main topics

- When multiple agents help (and when they only add cost)
- Patterns: orchestrator-worker, handoffs, parallel workers, reviewer/critic
- Planning: breaking a goal into subtasks
- Agent2Agent (A2A) protocol: agents talking to agents across systems

Detailed concepts

- Clear roles and narrow tools per agent
- Passing context between agents (what to share, what to hide)
- Parallel subagents and merging results
- Cost multiplication in multi-agent systems
- Agent Cards and task lifecycle in A2A

Learning tasks

1. Read "How we built our multi-agent research system" and note three lessons about orchestration and cost.
2. Read the OpenAI Agents SDK handoffs page and run the handoff example.
3. Read the A2A protocol overview and explain Agent Cards in your own words.
4. Draw your multi-agent design (agents, tools, messages, approval point) before writing code.

Resources

- Anthropic - How we built our multi-agent research system
<https://www.anthropic.com/engineering/multi-agent-research-system>
- OpenAI Agents SDK - Handoffs
<https://openai.github.io/openai-agents-python/handoffs/>
- A2A Protocol
<https://a2a-protocol.org/>
- CrewAI documentation
<https://docs.crewai.com/>

Practical project: Multi-Agent Research Team

A team of agents that produces a sourced research brief on a user's topic: an orchestrator plans sub-questions, researcher agents work in parallel with search tools, a writer drafts the brief, a reviewer checks it against a rubric, and a human approves before the final report is saved.

1. Draw the design: agents, tools, messages and the approval point.

2. Implement researcher and writer agents with narrow instructions.
3. Implement the orchestrator (planning + parallel research, max 4 workers).
4. Add the reviewer with max 2 revision rounds and a clear rubric.
5. Add human approval before saving/sending the final report, and log tokens per agent.

Bonus: Expose the research team as a service with an A2A-style Agent Card describing its skill.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Architecture diagram
3. Two example briefs with sources
4. Token/cost per agent for one run
5. README with setup steps, architecture diagram and example runs

Learning outcomes

- Chooses a multi-agent pattern that fits the problem
- Builds an orchestrator with specialised workers and a review step
- Explains how A2A complements MCP

Week 6: Browser and Code-Executing Agents

Objective

Give agents safe access to a real browser and to code execution in a sandbox, so they can collect data from websites and analyse it with code.

Main topics

- Browser agents: Playwright MCP and Browser Use
- Accessibility tree vs screenshots (how agents "see" a page)
- Computer-use agents that control a desktop, and their risks
- Code-executing agents and sandboxing (Docker, hosted sandboxes such as E2B)

Detailed concepts

- Page snapshots, element references and actions (click, type, navigate)
- Headless vs headed browsers
- Why generated code must never run on your own machine unprotected
- Resource limits: time, memory, network off
- Respecting website terms and robots rules

Learning tasks

1. Run Playwright MCP (`npx @playwright/mcp@latest`) in an MCP host and let an agent find and summarise a public page.
2. Read the Browser Use README and run its simplest example with your model provider.
3. Run a Python script inside Docker with `--network none` and a memory limit; confirm it cannot reach the internet.
4. Write a list of 5 actions your browser agent must never do without asking (e.g. submit forms, log in, pay).

Resources

- Playwright MCP (Microsoft)
<https://github.com/microsoft/playwright-mcp>
- Browser Use
<https://github.com/browser-use/browser-use>
- E2B sandboxes documentation
<https://docs.e2b.dev/>
- Docker - Get started
<https://docs.docker.com/get-started/>

Practical project: Web Data + Code Analysis Agent

An agent that collects public data from a website with a browser tool (Playwright MCP or Browser Use), saves it as CSV, then analyses it by writing Python that runs in a Docker sandbox with no network, returning a chart and a short written summary.

1. Pick a public page with a table or list (e.g. a public statistics page) and check its terms.
2. Build the browsing step with Playwright MCP or Browser Use and save clean CSV.
3. Build `run_python_sandboxed()` with no network, time and memory limits.
4. Let the agent write analysis code, run it in the sandbox and fix errors (max 3 attempts).
5. Add confirmation rules for forms, logins and downloads.

Bonus: Use a pre-built analysis image (pandas + matplotlib) and return the chart in the UI.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Chart output and summary for one real dataset
3. List of blocked/confirmation-required actions
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds an agent that navigates websites through a browser tool
- Runs model-written code only inside an isolated sandbox with limits
- Adds confirmation rules for risky browser actions

Week 7: Evaluation, Observability and Security

Objective

Measure how good your agent is with a test set and automated evals, trace every run, and report quality, cost and latency with numbers. Find and fix the main security risks of agents - prompt injection, tool misuse, excessive permissions, data leaks - using the OWASP Top 10 for Agentic Applications and red-team tests.

Main topics

- Why agents need evals: non-deterministic output, many steps
- Test sets, graders (code checks, LLM-as-judge, human review)
- Evaluating the path (trajectory) as well as the final answer
- OWASP Top 10 for Agentic Applications (2026)
- Direct and indirect prompt injection
- Least privilege, tool permissions and human approval

Detailed concepts

- Golden datasets and edge cases
- Deterministic assertions vs model-graded rubrics
- Pass rate, cost per task, latency (p50/p95)
- Goal hijack, tool misuse, identity and privilege abuse
- The "lethal trifecta": private data + untrusted content + a way to send data out
- Input/output guardrails and allow-lists

Learning tasks

1. Read "Demystifying evals for AI agents" and summarise the grader types it describes.
2. Read the OWASP Top 10 for Agentic Applications and map at least 5 risks to your own agent.
3. Install promptfoo and run its getting-started example.
4. Read "The lethal trifecta" article and check whether your agent has all three parts.
5. Create a 30-case test set for your agent (normal, edge and unanswerable cases) with expected results.

Resources

- Anthropic - Demystifying evals for AI agents
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- promptfoo documentation
<https://www.promptfoo.dev/docs/intro/>
- OWASP Top 10 for Agentic Applications for 2026
<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- OWASP Top 10 for LLM Applications
<https://genai.owasp.org/llm-top-10/>

Practical project: Evaluate, Trace and Secure Your Agent

Build a 30+ case eval set with code, rubric and trajectory checks, add Langfuse tracing, then red-team the agent against the OWASP agentic risks and fix the most serious issues in code.

1. 30+ test cases (normal, edge, refusal, multi-step) and an eval runner.
2. Tracing with Langfuse; report pass rate, cost/task, p50/p95 latency.
3. Threat model using the OWASP Top 10 for Agentic Applications.
4. 15+ attack cases including indirect prompt injection; run and record.
5. Fix in code (allow-lists, approvals, user-scoped data) and re-run evals + attacks.

Bonus: Run the eval set against two models and recommend one with numbers.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Eval report: pass rate, cost/task, p50/p95 latency
3. Before/after comparison for two fixes
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds and maintains a test set for an agent
- Runs automated evals and reads the results critically
- Threat-models an agent using the OWASP agentic risks
- Defends against prompt injection with design, not just prompts

Week 8: Production and Capstone

Objective

Package the agent as a real service with an API and UI, streaming, retries, caching and cost limits, deploy it, and present a portfolio-grade capstone.

Main topics

- Serving an agent with FastAPI and streaming responses
- Reliability: timeouts, retries with backoff, fallbacks, rate limits
- Cost control: caching, model routing, budgets
- Deployment with Docker and monitoring in production

Detailed concepts

- Server-sent events (SSE) for token streaming
- Idempotency and background jobs for long tasks
- Prompt caching and response caching
- Health checks, structured logs and alerts
- Writing a strong README and demo

Learning tasks

1. Follow the FastAPI tutorial up to "Request Body" and build a `/chat` endpoint that calls your agent.
2. Add streaming with `StreamingResponse` and show tokens appearing live in a simple web page.
3. Write a Dockerfile, build the image and run it locally with environment variables.
4. Deploy to a free host (for example Hugging Face Spaces or Render) and share the URL.

Resources

- FastAPI documentation
<https://fastapi.tiangolo.com/>
- Docker - Get started
<https://docs.docker.com/get-started/>
- Hugging Face Spaces documentation
<https://huggingface.co/docs/hub/spaces>
- Render documentation
<https://render.com/docs>

Practical project: Capstone: Ship a Production Agent

Turn your best agent into a deployed product that solves a real problem: a FastAPI service with streaming, a simple UI, retries and cost limits, Docker packaging, deployment to a free host, evals, security notes and a polished demo.

1. Write a one-page problem statement: user, problem, success measure.
2. Build the FastAPI service with `/health` and streaming `/chat`, plus a simple UI.

3. Add timeouts, retries with backoff, a fallback model, rate limits and a per-user budget.
4. Package with Docker and deploy; configure secrets on the host.
5. Run the full eval set against the deployed version and write the final README.

Bonus: Add a background-job endpoint for long tasks with job status polling.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Live URL
3. 3-5 minute demo video
4. Final README: architecture, evals, security, next steps
5. README with setup steps, architecture diagram and example runs

Learning outcomes

- Ships an agent as a deployed, documented web service
- Handles failures, limits and costs deliberately
- Presents a complete project with evals, security notes and a demo

Week 1: LLM Foundations and Structured Output

Objective

Call a large language model (LLM) from Python, control it with good prompts, and get reliable structured JSON back that your code can trust.

Main topics

- What an LLM is and what makes a program an "agent"
- Calling a model API from Python (OpenAI-compatible APIs, Gemini free tier, local models with Ollama)
- Prompting for agents: system prompts, roles, few-shot examples
- Structured outputs: JSON mode and validation with Pydantic

Detailed concepts

- Tokens, context window, temperature and max output tokens
- Messages array: system, user and assistant roles
- Keeping API keys in a `.env` file, never in code
- Validating model output and retrying when it is invalid
- Project setup with `uv` and a clean `pyproject.toml`

Learning tasks

1. Create a free Google AI Studio key and run your first chat completion from Python using the OpenAI-compatible endpoint.
2. Read the Gemini "OpenAI compatibility" page and note which parameters are supported.
3. Write the same request with three different system prompts and compare the answers in a short table.
4. Define a Pydantic model, ask the model for JSON that matches it, validate it, and add one automatic retry on validation errors.
5. Install Ollama, pull one small local model and point the same script at it by only changing the base URL and model name.

Resources

- Gemini API documentation
<https://ai.google.dev/gemini-api/docs>
- Gemini OpenAI compatibility
<https://ai.google.dev/gemini-api/docs/openai>
- uv - Python package and project manager
<https://docs.astral.sh/uv/>
- Pydantic documentation
<https://docs.pydantic.dev/latest/>

- Hugging Face AI Agents Course (free)
<https://huggingface.co/learn/agents-course>

Practical project: Smart Text Extractor CLI

A command-line tool that takes messy text (an email, job post or support message) and returns validated, structured JSON plus a one-line summary. It must work with the Gemini free tier and with a local Ollama model by changing only configuration.

1. Set up the project with uv, a `.env.example`, and a `config.py` that reads provider, base URL, key and model.
2. Define at least two Pydantic models (e.g. `SupportTicket`, `JobPost`) and a `--type` flag to choose one.
3. Write a system prompt with rules, the JSON shape and one example.
4. Validate the output with Pydantic and retry once with the error message.
5. Add 10 sample input files and a script that runs all of them and prints a results table.

Bonus: Add a `--compare` flag that runs the same input on two providers and shows the differences.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. `samples/` folder with 10 inputs and their outputs
3. Short note comparing results of two providers or models
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Can call any OpenAI-compatible model API from Python and switch providers with configuration only
- Writes clear system prompts with rules, examples and an output format
- Gets validated, typed data out of a model instead of free text

Week 2: Tool Calling and Your First Agent Loop

Objective

Give a model tools (Python functions) and build the agent loop yourself: the model picks a tool, your code runs it, the result goes back, until the task is done.

Main topics

- Function / tool calling: how a model asks your code to run a function
- The agent loop: reason, act, observe, repeat
- Stop conditions: final answer, step limit, errors
- Workflows vs agents, and when not to use an agent

Detailed concepts

- Tool schema: name, description and JSON-schema parameters
- Tool call IDs and sending tool results back with the `tool` role
- Parallel tool calls in one response
- Handling tool errors by returning them to the model
- Logging every step so you can debug the loop

Learning tasks

1. Read "Building effective agents" and write a half-page summary of the difference between workflows and agents.
2. Skim the ReAct paper abstract and figure 1; explain the reason, act, observe cycle in your own words.
3. Read the function-calling guide for your provider and run its example unchanged.
4. Add a second and third tool to the example, then print each step of the loop with the tool name and arguments.

Resources

- Anthropic - Building effective agents
<https://www.anthropic.com/engineering/building-effective-agents>
- Gemini API - Function calling
<https://ai.google.dev/gemini-api/docs/function-calling>
- ReAct: Synergizing Reasoning and Acting in Language Models (paper)
<https://arxiv.org/abs/2210.03629>
- Open-Meteo free weather API (no key)
<https://open-meteo.com/en/docs>

Practical project: Tool-Using Assistant From Scratch

A terminal assistant that answers questions using tools you wrote - weather (Open-Meteo), a calculator, current date/time and a small SQLite notes store - with your own agent loop, step limit and step-by-step logging.

1. Write each tool as a normal Python function with type hints and test it alone.
2. Write the tool schemas with clear descriptions.
3. Implement the agent loop with `max_steps`, error handling and logging.
4. Support parallel tool calls in one reply.
5. Create 15 test questions (including ones that need two tools and ones that need none) and record the steps taken.

Bonus: Generate tool schemas automatically from the function signatures and docstrings.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. `tests/questions.md` with 15 questions and the tools used for each
3. Log output of three runs in the README
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Can write tool schemas the model uses correctly
- Can build and debug an agent loop without any framework
- Knows when a fixed workflow is better than an autonomous agent

Week 3: Tool Design, Context Engineering and Memory

Objective

Design tools that agents use well, manage what goes into the context window, and give the agent short-term and simple long-term memory.

Main topics

- Designing good tools: naming, descriptions, inputs, outputs and errors
- Context engineering: choosing what the model sees on each call
- Short-term memory: conversation history, trimming and summarising
- Project context files: AGENTS.md and Agent Skills (SKILL.md)

Detailed concepts

- Few, well-scoped tools instead of many overlapping ones
- Token-efficient tool results (pagination, filtering, summaries)
- Context rot: quality drops as context grows
- Rolling window vs summary memory
- Storing facts about the user in a small database (long-term memory)

Learning tasks

1. Read "Writing effective tools for agents" and list five rules you will apply to your own tools.
2. Read "Effective context engineering for AI agents" and explain context rot in three sentences.
3. Read the AGENTS.md site and the Agent Skills specification; write an AGENTS.md for your own project.
4. Measure token usage of your agent over a 20-message conversation, then add summarisation and measure again.

Resources

- Anthropic - Writing effective tools for agents
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- Anthropic - Effective context engineering for AI agents
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- AGENTS.md - open format for guiding coding agents
<https://agents.md/>
- Agent Skills specification (SKILL.md)
<https://github.com/agentskills/agentskills>

Practical project: Personal Assistant With Memory

Upgrade your tool-using assistant: redesign the tools for clarity, add conversation compaction for long chats, and add long-term memory (facts about the user saved in SQLite) that survives restarts. Add an

AGENTS.md for the repository.

1. Review each tool against the "good tool" rules and improve names, descriptions and outputs.
2. Add `remember(fact, topic)` and `recall(topic)` tools backed by SQLite, keyed by user.
3. Load relevant memories into the system prompt at session start.
4. Add history compaction once the conversation passes a token threshold.
5. Measure tokens for a 20-message conversation before and after compaction.

Bonus: Add a `/forget` command that lets the user see and delete stored memories.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Before/after token measurement table
3. `AGENTS.md` at the repo root
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Designs tools with clear contracts and compact, useful results
- Keeps long conversations within budget without losing key facts
- Adds persistent memory that survives restarts

Week 4: RAG: Answering From Your Documents

Objective

Let an agent answer questions from your own documents by embedding them, storing them in a vector database, retrieving the best chunks and citing them.

Main topics

- Embeddings: turning text into vectors that capture meaning
- Vector databases: Chroma locally, pgvector in PostgreSQL
- The RAG pipeline: load, chunk, embed, store, retrieve, generate
- Answering with citations and saying "I don't know"

Detailed concepts

- Cosine similarity and nearest-neighbour search
- Chunk size and overlap, and why they matter
- Metadata (source file, page) for filtering and citations
- Hybrid search: keywords plus vectors
- Grounding: answer only from retrieved context

Learning tasks

1. Read the embeddings guide for your provider and embed ten sentences; find the most similar pair.
2. Follow the Chroma getting-started guide and store/query your ten sentences with metadata.
3. Try three chunk sizes (300, 800, 1500 characters) on the same documents and compare retrieval for five test questions.
4. Write a grounding prompt that forces citations like [source: file.pdf p.3] and refuses when the answer is not in the context.

Resources

- Gemini API - Embeddings
<https://ai.google.dev/gemini-api/docs/embeddings>
- Chroma documentation
<https://docs.trychroma.com/>
- pgvector - vector search for PostgreSQL
<https://github.com/pgvector/pgvector>
- Ollama (local embedding and chat models)
<https://ollama.com/>

Practical project: Document Q&A Bot With Citations

A chat app that answers questions about a set of real documents (for example the internship handbook, a university prospectus or product manuals) using RAG with a persistent Chroma index, citations and

honest "I don't know" answers, with a simple Streamlit or Gradio UI.

1. Collect 5-10 text-based PDFs or Markdown files and write an `ingest.py` that loads, chunks and indexes them with metadata.
2. Write `answer(question)` with retrieval, grounding prompt and citations.
3. Try three chunk sizes and pick one using 20 test questions with expected sources.
4. Build a minimal UI that shows the answer and the cited sources.
5. Include 5 unanswerable questions and show the bot refusing.

Bonus: Add hybrid search (keyword + vector) and compare results on names or codes.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. `eval/questions.csv` with question, expected source and result
3. Chunk-size comparison table in the README
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds a complete RAG pipeline from files to cited answers
- Chooses chunking and retrieval settings based on measurement, not guesses
- Prevents made-up answers with grounding rules

Week 5: Agentic RAG

Objective

Turn a fixed RAG pipeline into an agent that decides when and what to search, rewrites queries, combines several sources and checks its own answer.

Main topics

- From RAG pipeline to agentic RAG: retrieval as a tool
- Query rewriting and breaking a question into sub-questions
- Multiple sources: your documents, the web (Wikipedia), and user memory
- Self-checking: is the retrieved context enough to answer?

Detailed concepts

- Retrieval tools with filters (source, date, section)
- Multi-hop questions that need several searches
- Contextual retrieval: adding context to chunks before embedding
- Re-ranking the retrieved chunks
- Knowing when to stop searching

Learning tasks

1. Read "Introducing Contextual Retrieval" and list the two techniques it combines.
2. Turn your `retrieve()` function into a tool with parameters `query`, `source` and `k`, and let the agent call it.
3. Write 10 multi-hop questions (needing 2+ searches) and record how many searches your agent makes for each.
4. Add a Wikipedia search tool and a rule for when to use it instead of your documents.

Resources

- Anthropic - Introducing Contextual Retrieval
<https://www.anthropic.com/news/contextual-retrieval>
- Chroma - metadata filtering
<https://docs.trychroma.com/>
- MediaWiki Action API (Wikipedia search)
<https://www.mediawiki.org/wiki/API:Search>
- Hugging Face Agents Course - agentic RAG unit
<https://huggingface.co/learn/agents-course>

Practical project: Agentic Research Assistant

Turn the document bot into an agent: retrieval becomes tools (`search_docs` with filters, `search_wikipedia`), the agent rewrites queries, handles multi-hop questions, and stores user

preferences as long-term memory.

1. Wrap retrieval as a `search_docs(query, source, k)` tool.
2. Add a `search_wikipedia(query)` tool with a proper User-Agent header.
3. Write system rules: when to use which source, citation format, max 3 searches.
4. Log original question, rewritten queries and number of searches per run.
5. Test 10 multi-hop questions and 5 questions that need no search.

Bonus: Add contextual retrieval (prefix each chunk with its source/topic before embedding) and compare retrieval quality.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Table of 15 test questions with searches used and correctness
3. Log examples showing query rewriting
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds an agent that plans its own retrieval steps
- Handles multi-hop questions with query rewriting
- Combines private documents, public sources and memory safely

Week 6: Model Context Protocol (MCP)

Objective

Build an MCP server that exposes your tools and data to any MCP-compatible AI app, test it with the MCP Inspector, and call it from a client.

Main topics

- What MCP is and why it exists (one standard instead of custom integrations)
- MCP building blocks: tools, resources and prompts
- Transports: stdio for local, Streamable HTTP for remote servers
- The 2026-07-28 specification: stateless core, extensions (e.g. MCP Apps), stronger authorization

Detailed concepts

- Hosts, clients and servers
- Tool schemas generated from Python type hints
- Testing with the MCP Inspector
- Authorization for remote servers (OAuth-based)
- Security risks: untrusted servers, tool poisoning, over-broad permissions

Learning tasks

1. Read the MCP introduction and architecture pages; draw the host-client-server diagram.
2. Read the 2026-07-28 specification release post and list three changes.
3. Run the Python SDK quickstart server and open it in the MCP Inspector with `uv run mcp dev server.py`.
4. Connect your server to one MCP host you have (an IDE or desktop assistant that supports MCP) and use a tool from chat.

Resources

- Model Context Protocol - documentation
<https://modelcontextprotocol.io/>
- MCP Specification 2026-07-28
<https://modelcontextprotocol.io/specification/2026-07-28>
- MCP blog - The 2026-07-28 Specification
<https://blog.modelcontextprotocol.io/posts/2026-07-28/>
- MCP Python SDK
<https://github.com/modelcontextprotocol/python-sdk>
- MCP Inspector
<https://github.com/modelcontextprotocol/inspector>

Practical project: Your Own MCP Server

Expose your assistant's capabilities (notes/memory and document search) as an MCP server with the official Python SDK, test it in the MCP Inspector, and use it from a real MCP host and from a small Python client.

1. Create `server.py` with at least three tools (e.g. `search_docs`, `add_note`, `list_notes`) with type hints and docstrings.
2. Run it with `uv run mcp dev server.py` and test every tool in the Inspector.
3. Connect it to an MCP host you have and use it from chat.
4. Run it with Streamable HTTP and call a tool from a Python client.
5. Validate inputs and make sure no secrets are returned.

Bonus: Expose the documents as MCP resources and add one prompt template.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Inspector screenshots for each tool
3. Host configuration snippet in the README
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Explains MCP hosts, clients, servers, tools, resources and prompts
- Ships a working, documented MCP server
- Knows the main security risks of connecting agents to MCP servers

Week 7: Agent Frameworks and Human-in-the-Loop

Objective

Rebuild your agent with a current framework, use explicit state, checkpoints and human approval steps, and decide when a framework is worth it.

Main topics

- The current framework landscape: LangGraph, OpenAI Agents SDK, Pydantic AI, Google ADK, CrewAI
- Graphs and state machines for agents
- Checkpointing and resuming long-running agents
- Human-in-the-loop: pausing for approval or input

Detailed concepts

- Nodes, edges and conditional routing
- Typed state objects
- Threads and persistence
- Interrupts and resume commands
- Framework vs plain loop trade-offs

Learning tasks

1. Read the overview pages of LangGraph and the OpenAI Agents SDK, and fill a comparison table (model support, state, HITL, tracing, learning curve).
2. Do the LangGraph quickstart and add a conditional edge.
3. Add an interrupt before a risky tool call and resume it with an approval.
4. Write one page: "Would I use a framework for my project? Why or why not?"

Resources

- LangGraph documentation
<https://docs.langchain.com/oss/python/langgraph/overview>
- OpenAI Agents SDK (Python)
<https://openai.github.io/openai-agents-python/>
- Pydantic AI
<https://pydantic.dev/docs/ai/overview/>
- Google Agent Development Kit (ADK)
<https://google.github.io/adk-docs/>
- CrewAI documentation
<https://docs.crewai.com/>

Practical project: Rebuild With a Framework + Human Approval

Rebuild your research assistant with a current agent framework (LangGraph recommended; OpenAI Agents SDK, Pydantic AI or Google ADK allowed), with explicit state, a persistent checkpoint, and a human approval step before any action that changes data or sends something.

1. Choose a framework and justify it in the README.
2. Define the state schema and the graph/agents.
3. Plug in your existing tools (or your MCP server).
4. Add a SQLite (or other persistent) checkpoint and resume a conversation after restart.
5. Add an approval interrupt before `add_note` /any write action and resume with approve/reject.

Bonus: Stream intermediate steps (tool calls) to the terminal or UI while the agent runs.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Comparison: plain loop vs framework (lines of code, features, debugging)
3. Demo of approve and reject paths
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds a stateful agent graph with conditional routing
- Adds checkpoints and human approval to an agent
- Makes a reasoned framework choice for a project

Week 8: Multi-Agent Systems

Objective

Design and build a system where several specialised agents cooperate - an orchestrator with workers, handoffs between agents and a human approval step - and understand agent-to-agent protocols.

Main topics

- When multiple agents help (and when they only add cost)
- Patterns: orchestrator-worker, handoffs, parallel workers, reviewer/critic
- Planning: breaking a goal into subtasks
- Agent2Agent (A2A) protocol: agents talking to agents across systems

Detailed concepts

- Clear roles and narrow tools per agent
- Passing context between agents (what to share, what to hide)
- Parallel subagents and merging results
- Cost multiplication in multi-agent systems
- Agent Cards and task lifecycle in A2A

Learning tasks

1. Read "How we built our multi-agent research system" and note three lessons about orchestration and cost.
2. Read the OpenAI Agents SDK handoffs page and run the handoff example.
3. Read the A2A protocol overview and explain Agent Cards in your own words.
4. Draw your multi-agent design (agents, tools, messages, approval point) before writing code.

Resources

- Anthropic - How we built our multi-agent research system
<https://www.anthropic.com/engineering/multi-agent-research-system>
- OpenAI Agents SDK - Handoffs
<https://openai.github.io/openai-agents-python/handoffs/>
- A2A Protocol
<https://a2a-protocol.org/>
- CrewAI documentation
<https://docs.crewai.com/>

Practical project: Multi-Agent Research Team

A team of agents that produces a sourced research brief on a user's topic: an orchestrator plans sub-questions, researcher agents work in parallel with search tools, a writer drafts the brief, a reviewer checks it against a rubric, and a human approves before the final report is saved.

1. Draw the design: agents, tools, messages and the approval point.

2. Implement researcher and writer agents with narrow instructions.
3. Implement the orchestrator (planning + parallel research, max 4 workers).
4. Add the reviewer with max 2 revision rounds and a clear rubric.
5. Add human approval before saving/sending the final report, and log tokens per agent.

Bonus: Expose the research team as a service with an A2A-style Agent Card describing its skill.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Architecture diagram
3. Two example briefs with sources
4. Token/cost per agent for one run
5. README with setup steps, architecture diagram and example runs

Learning outcomes

- Chooses a multi-agent pattern that fits the problem
- Builds an orchestrator with specialised workers and a review step
- Explains how A2A complements MCP

Week 9: Browser and Code-Executing Agents

Objective

Give agents safe access to a real browser and to code execution in a sandbox, so they can collect data from websites and analyse it with code.

Main topics

- Browser agents: Playwright MCP and Browser Use
- Accessibility tree vs screenshots (how agents "see" a page)
- Computer-use agents that control a desktop, and their risks
- Code-executing agents and sandboxing (Docker, hosted sandboxes such as E2B)

Detailed concepts

- Page snapshots, element references and actions (click, type, navigate)
- Headless vs headed browsers
- Why generated code must never run on your own machine unprotected
- Resource limits: time, memory, network off
- Respecting website terms and robots rules

Learning tasks

1. Run Playwright MCP (`npx @playwright/mcp@latest`) in an MCP host and let an agent find and summarise a public page.
2. Read the Browser Use README and run its simplest example with your model provider.
3. Run a Python script inside Docker with `--network none` and a memory limit; confirm it cannot reach the internet.
4. Write a list of 5 actions your browser agent must never do without asking (e.g. submit forms, log in, pay).

Resources

- Playwright MCP (Microsoft)
<https://github.com/microsoft/playwright-mcp>
- Browser Use
<https://github.com/browser-use/browser-use>
- E2B sandboxes documentation
<https://docs.e2b.dev/>
- Docker - Get started
<https://docs.docker.com/get-started/>

Practical project: Web Data + Code Analysis Agent

An agent that collects public data from a website with a browser tool (Playwright MCP or Browser Use), saves it as CSV, then analyses it by writing Python that runs in a Docker sandbox with no network, returning a chart and a short written summary.

1. Pick a public page with a table or list (e.g. a public statistics page) and check its terms.
2. Build the browsing step with Playwright MCP or Browser Use and save clean CSV.
3. Build `run_python_sandboxed()` with no network, time and memory limits.
4. Let the agent write analysis code, run it in the sandbox and fix errors (max 3 attempts).
5. Add confirmation rules for forms, logins and downloads.

Bonus: Use a pre-built analysis image (pandas + matplotlib) and return the chart in the UI.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Chart output and summary for one real dataset
3. List of blocked/confirmation-required actions
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds an agent that navigates websites through a browser tool
- Runs model-written code only inside an isolated sandbox with limits
- Adds confirmation rules for risky browser actions

Week 10: Evaluation and Observability

Objective

Measure how good your agent is with a test set and automated evals, trace every run, and report quality, cost and latency with numbers.

Main topics

- Why agents need evals: non-deterministic output, many steps
- Test sets, graders (code checks, LLM-as-judge, human review)
- Evaluating the path (trajectory) as well as the final answer
- Tracing with Langfuse and OpenTelemetry GenAI conventions

Detailed concepts

- Golden datasets and edge cases
- Deterministic assertions vs model-graded rubrics
- Pass rate, cost per task, latency (p50/p95)
- Regression testing before every change
- Traces, spans and sessions

Learning tasks

1. Read "Demystifying evals for AI agents" and summarise the grader types it describes.
2. Install promptfoo and run its getting-started example.
3. Create a 30-case test set for your agent (normal, edge and unanswerable cases) with expected results.
4. Add Langfuse tracing (cloud free tier or self-hosted) and find your slowest and most expensive run.

Resources

- Anthropic - Demystifying evals for AI agents
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- promptfoo documentation
<https://www.promptfoo.dev/docs/intro/>
- Langfuse documentation
<https://langfuse.com/docs>
- OpenTelemetry semantic conventions for generative AI
<https://opentelemetry.io/docs/specs/semconv/gen-ai/>

Practical project: Evaluate and Trace Your Agent

Build a proper evaluation and observability setup for your most advanced agent so far: a 30+ case test set, automated evals (code checks + LLM rubric + trajectory checks), tracing with Langfuse, and a report with quality, cost and latency.

1. Write 30+ test cases: normal, edge, refusal and multi-step.
2. Create an eval runner (promptfoo or pytest) with code checks and rubric checks.
3. Record tool calls per run and add trajectory assertions.
4. Add Langfuse tracing and find the slowest and most expensive runs.
5. Fix two problems you found and show before/after numbers.

Bonus: Run the eval set against two models and recommend one with numbers.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Eval report: pass rate, cost/task, p50/p95 latency
3. Before/after comparison for two fixes
4. README with setup steps, architecture diagram and example runs

Learning outcomes

- Builds and maintains a test set for an agent
- Runs automated evals and reads the results critically
- Uses traces to find slow, expensive or failing steps

Week 11: Agent Safety and Security

Objective

Find and fix the main security risks of agents - prompt injection, tool misuse, excessive permissions, data leaks - using the OWASP Top 10 for Agentic Applications and red-team tests.

Main topics

- OWASP Top 10 for Agentic Applications (2026)
- Direct and indirect prompt injection
- Least privilege, tool permissions and human approval
- Guardrails, secrets handling and red-teaming

Detailed concepts

- Goal hijack, tool misuse, identity and privilege abuse
- The "lethal trifecta": private data + untrusted content + a way to send data out
- Input/output guardrails and allow-lists
- Memory and context poisoning
- Audit logs for agent actions

Learning tasks

1. Read the OWASP Top 10 for Agentic Applications and map at least 5 risks to your own agent.
2. Read "The lethal trifecta" article and check whether your agent has all three parts.
3. Write 15 attack prompts (including a web page or document with hidden instructions) and run them against your agent.
4. Run promptfoo red-teaming against your agent and fix the top two findings.

Resources

- OWASP Top 10 for Agentic Applications for 2026
<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- OWASP Top 10 for LLM Applications
<https://genai.owasp.org/llm-top-10/>
- promptfoo - LLM red teaming
<https://www.promptfoo.dev/docs/red-team/>
- Simon Willison - The lethal trifecta for AI agents
<https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>

Practical project: Red-Team and Harden Your Agent

Attack your own agent and fix what breaks: map it to the OWASP agentic risks, run 15+ attack cases including indirect prompt injection from documents and web pages, then add allow-lists, approval gates, audit logs and guardrails.

1. Write a threat-model table using the OWASP Top 10 for Agentic Applications.

2. Create 15+ attack cases (direct, indirect via documents/web, data access, dangerous tools).
3. Run them manually and with promptfoo red-teaming; record results.
4. Add defences in code: allow-lists, approval gates, user-scoped data access, output checks.
5. Re-run the attacks and add them to the eval set.

Bonus: Show the lethal-trifecta analysis for your agent and remove one leg for sensitive tasks.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Threat model table
3. Attack results before/after
4. Audit log sample
5. README with setup steps, architecture diagram and example runs

Learning outcomes

- Threat-models an agent using the OWASP agentic risks
- Defends against prompt injection with design, not just prompts
- Runs red-team tests and documents fixes

Week 12: Production and Capstone

Objective

Package the agent as a real service with an API and UI, streaming, retries, caching and cost limits, deploy it, and present a portfolio-grade capstone.

Main topics

- Serving an agent with FastAPI and streaming responses
- Reliability: timeouts, retries with backoff, fallbacks, rate limits
- Cost control: caching, model routing, budgets
- Deployment with Docker and monitoring in production

Detailed concepts

- Server-sent events (SSE) for token streaming
- Idempotency and background jobs for long tasks
- Prompt caching and response caching
- Health checks, structured logs and alerts
- Writing a strong README and demo

Learning tasks

1. Follow the FastAPI tutorial up to "Request Body" and build a `/chat` endpoint that calls your agent.
2. Add streaming with `StreamingResponse` and show tokens appearing live in a simple web page.
3. Write a Dockerfile, build the image and run it locally with environment variables.
4. Deploy to a free host (for example Hugging Face Spaces or Render) and share the URL.

Resources

- FastAPI documentation
<https://fastapi.tiangolo.com/>
- Docker - Get started
<https://docs.docker.com/get-started/>
- Hugging Face Spaces documentation
<https://huggingface.co/docs/hub/spaces>
- Render documentation
<https://render.com/docs>

Practical project: Capstone: Ship a Production Agent

Turn your best agent into a deployed product that solves a real problem: a FastAPI service with streaming, a simple UI, retries and cost limits, Docker packaging, deployment to a free host, evals, security notes and a polished demo.

1. Write a one-page problem statement: user, problem, success measure.
2. Build the FastAPI service with `/health` and streaming `/chat`, plus a simple UI.

3. Add timeouts, retries with backoff, a fallback model, rate limits and a per-user budget.
4. Package with Docker and deploy; configure secrets on the host.
5. Run the full eval set against the deployed version and write the final README.

Bonus: Add a background-job endpoint for long tasks with job status polling.

Deliverables

1. Public GitHub repository URL and demo (live URL or screen-recorded video) - mandatory
2. Live URL
3. 3-5 minute demo video
4. Final README: architecture, evals, security, next steps
5. README with setup steps, architecture diagram and example runs

Learning outcomes

- Ships an agent as a deployed, documented web service
- Handles failures, limits and costs deliberately
- Presents a complete project with evals, security notes and a demo